
Trial Exam I - Beta
11 December, 2013

Duration: 3.5 hours

3 questions

All questions should be attempted

Problem 1

Zigging

Input File: *zig.in*
Output File: *zig.out*

Time and Memory Limits: 1 second, 20 MB

The annual Macquarie Zigging championship is approaching. It is rumoured that thousands are turning up this year to view the spectacle. In your quest for fitness, you have entered yourself into the race too. A zigging race takes place in a long and narrow forest consisting of N evenly-spaced trees in a single row, lined up from West to East. At each tree along this forest is a checkpoint. Each checkpoint can give you a specific token, identified by an integer between 1 and K , inclusive.

The goal of zigging is to travel through the forest from checkpoint to checkpoint and collect all the tokens from 1 to K as quickly as possible. Collecting a token from a checkpoint takes no time; you must simply run past the checkpoint without needing to stop. Several different checkpoints may give you the same token. While token 1 may be collected at any time, the challenge of zigging arises from the rule that to collect token $i+1$, you must already have token i . Once you have token K , you have completed the race. Finally, bureaucratic occupational health and safety regulations insist that competitors only travel through the forest in an Eastwards direction. Any attempt to head West will lead to instant disqualification.

To increase your chances of winning the championship, you have devised a plan. You have obtained a map of the forest from the organisers, detailing precisely which token is available from each of the checkpoints. Your plan is to choose a checkpoint to start at which ensures you can get all K tokens as quickly as possible. When the race starts, you will arrive at that starting point and head East at a constant pace, collecting the tokens you require to finish.

For example, consider the forest below. The token number available from each checkpoint is shown on each tree. You are required to collect tokens 1 to 4:

In this scenario, four possible methods of collecting the four tokens are shown above, a black circle indicating where tokens are obtained. The shortest way to collect all four tokens is shown by the last of these lines. The runner should begin at tree 7 to collect token 1, and head East to collect the remaining tokens from trees 9, 11 and 12. This route passes 6 checkpoints in total, and so the answer to this scenario is 6.

Your task is to, given a map of the forest, determine the fewest number of checkpoints you must pass in order to collect all the tokens from 1 to K inclusive. You are guaranteed that it will always be possible to finish the race without cheating.

Input

The first line of the input file will contain two integers N K separated by a single space. N represents the number of checkpoints in the forest ($1 \leq N \leq 1\,000\,000$), while K is the number of tokens you are required to collect ($1 \leq K \leq 1000$, $K \leq N$).

Following this will be N lines, listing the token available at each checkpoint, in order from West to East. Each line will contain a single integer, between 1 and K inclusive.

For 30% of the available marks, $1 \leq N \leq 1000$.

Output

Your output file should contain a single integer, giving the fewest number of checkpoints you must pass to collect all K tokens.

Sample Input

```
13 4
2
1
4
2
4
1
1
3
2
4
3
4
2
```

Sample Output

```
6
```

Scoring

The score for each input scenario will be 100% if the correct answer is written to the output file, and 0% otherwise.

Problem 2

Beading

Input File: *beading.in*

Output File: *beading.out*

Time and Memory Limits: 0.2 seconds, 256 MB

You are a mathematician, and as a mathematician you have many friends, many of whom are similarly mathy to yourself. You want to give your best mathematician friend a special friendship bracelet as a celebration of your shared mathiness. As your best friend is also a computer scientist, you plan to make her a beaded bracelet where the beads form a complete infinite binary tree.

You have put a great deal of thought into this design. It will start with one bead at the top of the bracelet, which will then be connected by coloured thread to two beads immediately below. Each of these two beads will be connected to two beads in the row below, in usual binary tree fashion. There is also a thread that runs across each horizontal row of beads, connecting each bead to any bead on its immediate left or right.

Being a cryptographer, you want to be able to use this bracelet to send your friend secret number messages. You will send your friend two strings of characters representing paths along the threads in the bracelet, with which she can identify two specific beads in the bracelet. Each character is an instruction to step to the next bead in a particular direction, forming a path starting at the single bead at the top of the bracelet. The possible directions are:

- 1 - downwards left
- 2 - downwards right
- L - left along the same row
- R - right along the same row
- U - up

Once the two beads are identified, your friend will be able to work out the secret number, as it's simply the shortest distance between the two beads along the threads of the bracelet, counting each continuous section of thread between two adjacent beads as one unit. Too easy! You can't wait to present your friend with the beaded binary bracelet, and to secretly share all sorts of numbers with her.

Find a way to work out the secret number from two given character strings.

Input

The first line is a sequence of characters of maximum length 100,000: the (valid) instructions for reaching the first bead.

The second line is a sequence of characters of maximum length 100,000: the (valid) instructions for reaching the second bead.

Output

One integer: the smallest number of thread-segment steps between the two beads.

Sample Input 1

111RRRRRRR
222

Sample Output 1

0

Sample Input 2

221LU
12L2

Sample Output 2

3

Sample Input 3

11111
222222

Sample Output 3

10

Subtask

- For Subtask 1 (20 points), the paths never go below row 10.
- For Subtask 2 (20 points), the paths never go below row 50.
- For Subtask 3 (30 points), the paths never go below row 1000.
- For Subtask 4 (30 points), no further restrictions apply.

Problem 3

Catacombs and Treasure!

Input File: *catacombs.in*
Output File: *catacombs.out*

Time and Memory Limits: 2 seconds, 256 MB

You are exploring a treasure-filled catacomb, which can be defined recursively as follows:

A **subcatacomb** is a network of caves. Every subcatacomb has a leftmost cave and a rightmost cave (these may be the same). Every subcatacomb has a representation as a string.

Every subcatacomb is one of the following:

- An empty cave, written as “o”.
- A cave filled with treasure, written as “x”.
- Two subcatacombs C_L and C_R joined left to right, so that C_L ’s rightmost cave is connected to C_R ’s leftmost cave. (The leftmost cave of this subcatacomb is C_L ’s leftmost cave; the rightmost cave is C_R ’s rightmost cave.)
This is written as “ $\mathbf{S}C_L C_R\mathbf{\#}$ ”.

- Two subcatacombs C_1 and C_2 joined in parallel. The leftmost cave is a new cave, r_L , which is connected to the leftmost caves of C_1 and C_2 . Similarly, the rightmost cave is a new cave, r_R .
This is written as “ $\mathbf{P}r_L|C_1 C_2|r_R\mathbf{\#}$ ”.

Finally, the overall catacomb is itself a subcatacomb, plus an *exit* to the outside world from the leftmost cave.

The above diagram shows an example catacomb (corresponding to Sample Input 1 further below).

Rooms with treasure are very crowded and hard to pass through. Luckily, every treasure cave has a *path to the exit* which only passes through empty caves. (Call this the “treasure-reachability condition”.)

You love treasure and want to fill the catacomb with your personal collection! Your task here is to turn as many empty caves into treasure caves as you can (leaving the existing treasure caves as they are), such that the treasure-reachability condition still holds.

Input

There will be a single line of input, containing the string representation of the catacomb. This will be between 1 and 100 000 characters long. The only characters appearing in this line will be “o”, “x”, “P”, “S”, “|”, and “#”.

Output

The first line of output should consist of a single integer: the maximum number of treasure caves you can have such that every treasure cave has a path to the exit going only through empty caves.

The second line should describe how the catacomb might look once it has this many treasure caves, using the same format as the input. If there is more than one optimal assignment of treasure caves, any will do.

Sample Input 1

```
Po|Px|Sxo#Soo#|o#Soo#|o#
```

Sample Output 1

```
3
```

```
Po|Px|Sxo#Sox#|o#Soo#|o#
```

Sample Input 2

```
Po|SPo|oo|o#Px|oo|o##Po|Sxo#Po|ox|o#|o#|o#
```

Sample Output 2

```
7
```

```
Po|SPo|xx|o#Px|ox|o##Po|Sxx#Po|ox|o#|o#|o#
```

Subtasks

- For Subtask 1 (30 points), there are at most 20 caves in the catacomb.
- For Subtask 2 (40 points), there is no treasure in any of the caves.
- For Subtask 3 (30 points), no further constraints apply.

Scoring

Your score for each test case will be 100% if your program provides the optimal number of treasure caves and an optimal arrangement, 80% if it provides the optimal number of treasure caves on the first line but the second line is missing, malformed, suboptimal or invalid, and 0% otherwise.